

Mientras la microprogramación proporciona una alternativa al diseño tradicional de la máquina de estado, la microprogramación tiene en cuenta también un estilo de programar que es más bajo en nivel que el lenguaje ensamblador. En este proyecto usted construirá una trayectoria de datos sencillo, controlado por una estructura micro programado. Este proyecto es influenciado grandemente por Tanenbaum's "Structured Computer Organization", tercera edición.

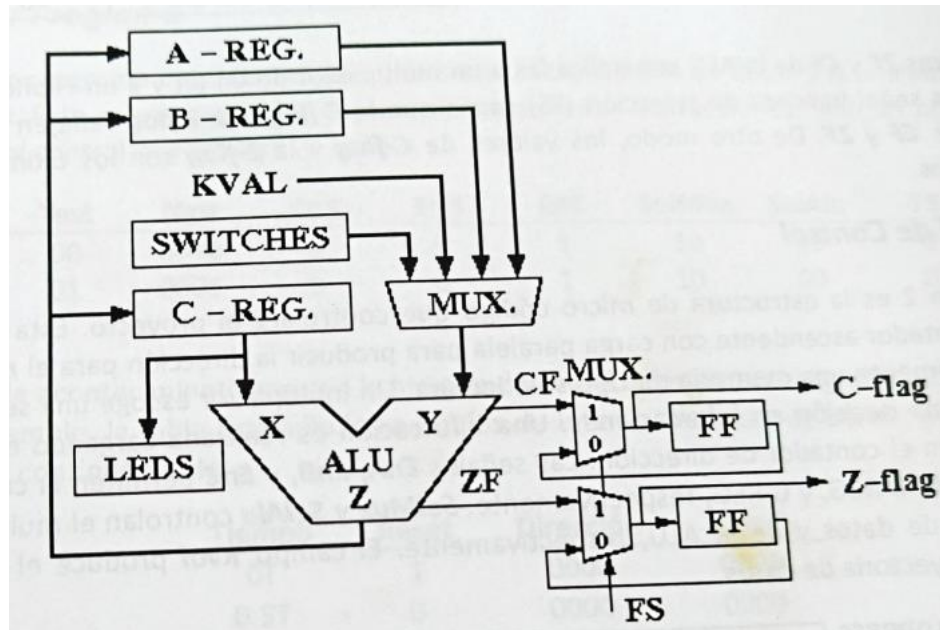


Ilustración 1 Proyecto Trayectoria de Datos

La trayectoria de Datos

La ilustración 1 es la trayectoria de datos para éste proyecto. Registros A, B, y C son todos registros síncronos idénticos de cuatro bits, cada uno realiza una carga paralela cuando se habilitan. El registro C produce la entrada X a la unidad aritmética-lógica (ALU) y las salidas a LED. La caja de interruptores es la entrada externa y el bus KVAL viene de la lógica de control. El multiplexor produce la entrada Y al ALU.

Las tablas siguientes resumen la conducta del multiplexor y de la ALU. La señal Z-flag de la ALU es alta siempre que la salida de la ALU sea "0000". Al realizar la adición, CF indica acarreo de salida desde el bit más significativo. Al realizar la sustracción, el complemento a dos de la entrada Y es formado y sumado a X; en este caso la señal de CF indica **PRESTAMO** y puede ser producido invirtiendo el acarreo de salida del sumador. La operación AND es "bitwise", entre los bits correspondientes de X y Y.

Comportamiento del Multiplexor	
SEL-MUX	OPERACIÓN
00	$Y = A\text{-REG}$
01	$Y = B\text{-REG}$
10	$Y = KVAL$
11	$Y = \text{INTERRUPTORES}$

Comportamiento de la ALU			
SEL-ALU	OPERACIÓN	CF	ZF
00	$Z = X + Y$	Acarreo	$Z = 0?$
01	$Z = X - Y$	Préstamo	$Z = 0?$
10	$Z = Y$	0	$Z = 0?$
11	$Z = X \text{ AND } Y$	0	$Z = 0?$

Las banderas **ZF** y **CF** de la ALU son aplicadas a un multiplexor de un bit y a un Flip-flop. Cuando son alta, la señal bandera de selección (FS) causa que la **C-flag** y **Z-flag** reflejen los valores actuales de **CF** y **ZF**. De otro modo, los valores de **C-flag** y la **Z-flag** son los últimos valores almacenados.

La Unidad de Control

La ilustración 2 es la estructura de micro código que controlará el proyecto. Esta estructura utiliza un contador ascendente con carga paralela para producir la dirección para la **microstore**, que es básicamente una memoria de tipo de sólo lectura. Un multiplexor escoge una señal fuente para tomar una decisión en bifurcaciones. Una bifurcación es realizada cargando el valor del campo **Natt** en el contador de dirección. Las señales **EnA**, **EnB**, y **EnC** permiten la carga de los registros A-REG, B-REG y C-REG, respectivamente. **SelMux** y **SelAlu** controlan el multiplexor de la trayectoria de datos y de la ALU, respectivamente. El campo **Kval** produce el bus **KVAL**, pasado a la trayectoria de datos.

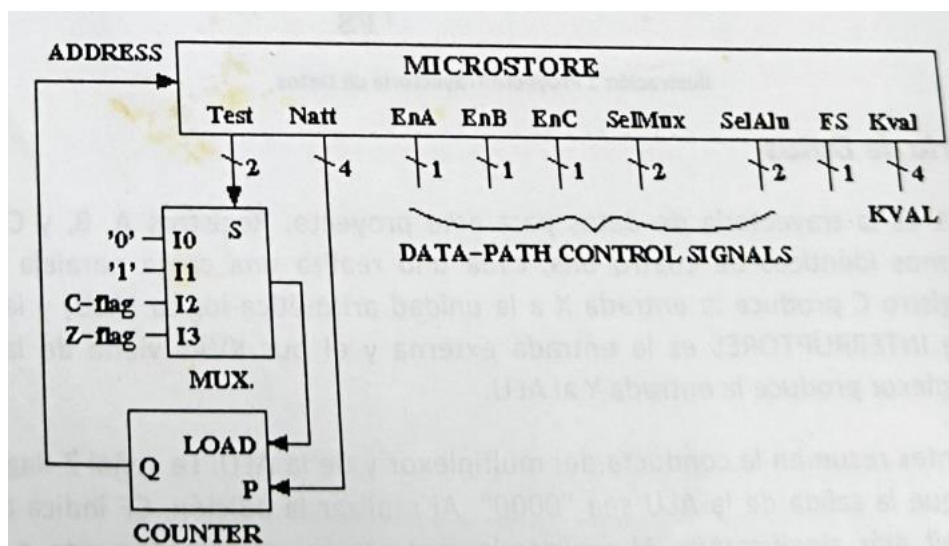


Ilustración 2 Estructura Micro código

Una señal externa asíncrona de **reset** causa que todos los registros y los Flipflop se borren y hace que el contador de dirección se ponga en cero. La tabla siguiente resume el comportamiento de bifurcación, basado en el campo **Test** de **microstore**. Las señales **C-flag** y **Z-flag**, producido por la ALU indica un resultado acarreo/préstamo o un resultado de cero respectivamente.

Test	Condición de Bifurcación
00	Nunca Bifurques
01	Siempre Bifurques
10	Bifurcación si C-flag es alto
11	Bifurcación si Z-flag es alto

Un Primer Programa

Este primer programa debe dar un sabor de cómo la trayectoria de datos y la unidad de control interactúan. Este es un programa muy corto, con sólo dos instrucciones. Sólo las primeras dos entradas del control son listadas:

Dirección	Test	Natt	EnA	EnB	EnC	SelMux	SelAlu	FS	Kval
0000	0	0000	0	0	1	10	10	0	0000
0001	1	0001	0	0	1	10	00	0	0001

Una tabla de acontecimiento resume la historia de la ejecución de un programa. Como siempre, dado un ejemplo, la tabla lista sólo los artículos de interés. Después de borrar, el registro C se incrementa continuamente.

Tiempo	Reset	Dirección	Reg-C
0T	1	0000	0000
0.5T	0	0000	0000
1T+	0	0001	0000
2T+	0	0001	0001
3T+	0	0001	0010
4T+	0	0001	0011

Micro Código Simbólico

El contenido de **microstore** es bastante denso y no tan fácil de leer. Para ayudar a escribir micro código nosotros utilizamos un lenguaje simbólico. La forma simbólica descrita aquí simplemente es una conveniencia para describir las operaciones en la trayectoria de datos. La validez de una declaración depende de lo que el hardware sea capaz de hacer. Por lo que las referencia, **Tanenbaum's "Structured Computer Organization"** así como el **Mano y Kime, "Logic and Computer Design Fundamentals"** quizás sean útiles para lecturas complementarias.

Una instrucción incluye una parte opcional de ALU, una parte opcional de bifurcación, y un valor constante arbitrario. El símbolo $\leftarrow$ implica la carga de un registro y es utilizado con una operación de la ALU o pasando simplemente un

valor a la ALU. Las cosas que pueden ser referidas incluyen los registros A, B y C así como una entrada constante dada o una externa (SW).

- + significa adición
- - significa sustracción
- & significa bitwise AND

Una restricción presentada por la arquitectura es que las operaciones que involucran dos cosas debe involucrar primero el registro C. Considere lo siguiente:

$A \leftarrow C + A$

La declaración de arriba implica lo siguiente:

- Ponga el multiplexor de la trayectoria de datos para pasar el valor del registro A
- Ponga la ALU para realizar la adición
- Habilite sólo el registro A para cargar un nuevo valor

Para describir la capacidad de cargar a la ALU múltiples registros simultáneamente, una lista de registros separada por comas es colocada a lado izquierdo de la flecha.

$A,B,C \leftarrow C - A$

A veces es útil especificar una operación de la ALU sin asignar el resultado a cualquier registro. Para comparar los valores en los registros C y B realice la sustracción. Con valores iguales un resultado de cero ocurre, con el segundo número más alto en valor, un préstamo ocurre. La bandera Z puede ser utilizada para condicionar un salto, que discutiremos luego.

$C - B$

Cada instrucción de micro código puede realizar también bifurcación incondicional o condicional. Tres condiciones de bifurcación pueden ser descritas. Las instrucciones siguientes de bifurcación asumen que **Z-flag** y **C-flag** con iguales a **ZF** y **CF**, respectivamente (**FS** es '1'). Si ninguna condición de bifurcación es descrita en una instrucción, se asume el caso de ninguna bifurcación.

- Goto address - Un salto condicional
- If Z-flag goto address - Bifurca si la Z-flag de la ALU es alto (resultado 0)
- If C-flag goto address - Bifurca si la C-flag de la ALU es alto

Tales declaraciones de bifurcación implican que **FS** es "0" para la instrucción actual y '1' para la instrucción previa. Gran cuidado se debe tener al aplicar esta opción como bifurcación, puede haberlo llevado a la instrucción actual.

Programa de Ejemplo

Para escribir un programa simbólico en micro código, utilizamos una declaración **ORG** para poner la dirección inicial del **microstore**. Las etiquetas son nombres simbólicos utilizados para referirnos a direcciones del **microstore**. Los caracteres siguientes tienen el significado especial:

% Implica que un valor es dado en binario
 \$ Implica que un valor es dado en hexadecimal
 ; Indica el comienzo de una cadena de comentario
 * Representa la dirección de la instrucción actual

El siguiente es la forma simbólica del primer programa de ejemplo, dado más temprano. La etiqueta **Start** es reservada y se refiere al principio del programa.

; Un primer programa de ejemplo

```

      ORG $0
Start: C ← $1           ; inicie el conteo en 1
Loop: C ← C + $1        ; incremente
      C - $4, if Z-flag goto Start ; compare contra el límite
      goto Loop
  
```

Un listado de archivo simbólico agrega una columna en el extremo izquierdo de los valores secuenciales de dirección. Tal lista simbólica es útil para llenar una tabla de acontecimiento o para interpretar los resultados de la simulación.

; Un contador repetitivo, uno a cuatro

```

                ORG $0
%0000      Start: C ← $1           ; inicie la cuenta en 1
%0001      Loop: C ← C + $1        ; incremente
%0010      C - $4, if Z-flag goto Start ; Compare contra el límite
%0011      goto Loop
  
```

La table siguiente de acontecimiento resume la ejecución del programa. Para acentuar las características de los registros y la lógica combinacional, seguimos una convención sencilla. Mientras una entrada vacía para la entrada indica el valor previo (no cambio), una entrada vacía para un registro o cerrojo (Latch) indica que el dispositivo indica que el dispositivo contiene el valor previamente salvado. Escribir el valor de un registro o cerrojo (Latch) para cada carga de registro o tiempo el cerrojo es transparente, incluso si el nuevo valor es igual que el valor previo. Siempre escriba los valores de salida de las señales de lógica combinacional.

Tiempo	Reset	Dirección	Reg-C	ZF	Z-flag
0T	1	0000	0000	0	0
0.5T	0			0	
1T+		0001	0001	0	
2T+		0010	0010	0	0
3T+		0011		0	

4T+		0001		0	
5T+		0010	0011	0	0
6T+		0011		0	
7T+		0001		0	
8T+		0010	0100	1	1
9T+		0000		0	
10T+		0001	0001	0	Repite el programa

Tenga presente que los registros son actualizados en el flanco positivo del reloj y de ahí que los registros aparecen retrasados en la tabla de acontecimiento. En los tiempos 0T y 0.5T la dirección refiere a una instrucción que carga \$1 en el registro C, pero que el valor no aparece en el registro C hasta el tiempo 1T+.

Sin embargo, ponga especial atención a las señales CF y ZF del ALU que son simples señales de lógica combinatorial y reflejan las condiciones actuales en el ALU. Con la señal selección-bandera (FS) alta, Z-flag y C-flag son iguales a ZF y CF, respectivamente. En el tiempo 8T+ la dirección refiere a la instrucción de bifurcación condicional. Note que la Z-flag es afectada inmediatamente sin ningún retardo observable.

En la compilación del micro código para este segundo ejemplo encontramos las direcciones actuales asignadas a las etiquetas **Start** y **Loop**. Lo que sigue es el correspondiente micro código, seguido por la lista de valores asignadas a las etiquetas. Note que ceros son insertados en los campos que no interesan.

Dirección	Test	Natt	EnA	EnB	EnC	SelMux	SelAlu	FS	Kval
0000	00	0000	0	0	1	10	10	0	0001
0001	01	0000	0	0	1	10	00	0	0001
0010	11	0000	0	0	0	10	01	1	0100
0011	01	0001	0	0	0	00	00	0	0000

Resumen de Símbolos

Start = 0000

Loop = 0001

Este tercer ejemplo ilustra algunas características en el hardware. **Este será el programa que usted demostrará y presentará en su reporte.** Para demostrar su código, primeramente tendrá los interruptores de entrada en \$00 para demostrar el retardo de la bifurcación y entonces cambiará los interruptores para permitir al programa que proceda a su siguiente parte.

; Código de demostración del proyecto

```

                ORG $0
%0000          Start:      C ← $F
%0001          GetSW:      A,B ← SW
%0010                          B,C ← C – A, if Z-flag* goto Start
%0011                          C ← A
%0100          Top:        C ← C + 1
%0101                          C – B, if C-flag goto Top
%0110                          goto Start

```

Implementación

La tabla siguiente resume las entradas y salidas generales del sistema. La frecuencia de entrada de reloj será dividida para producir un reloj del sistema en Hercios. El contenido del registro C-REG debe ser asignado a LED's para demostrar los resultados. Utilice LED's para demostrar el valor de la dirección del contador también.

Señales Generales del Sistema		
Nombre	Dirección	Descripción
SW	In	Entrada desde los interruptores
Clock	In	Reloj desde el Oscilador
Reset	In	Reset asíncrono
ADDx	Out	Valor del contador de direcciones
LEDS	Out	Contenidos del C-REG

Los componentes a utilizar en el proyecto son los siguientes:

- Tarjeta Tang Nano 9K o Nexys-3 o Basys-3 o Mimas V2
- Diodos LED's (si es necesario)
- Software Gowin o Xilinx ISE
- Cable de programación
- Computadora personal

Asuma que la primera instrucción a ejecutar es siempre en dirección \$0. Para su informe usted incluirá el código de demostración del proyecto, micro código y simulación correspondiente. Resuma los resultados de la simulación en una tabla de eventos (acontecimiento) y adjunte los resultados verdaderos al final de su informe. En la tabla de acontecimiento liste los valores para el tiempo, y para las señales siguientes: Reset, Dirección, REG-A, REG-B, REG-C, Z-flag y C-flag. Para hacer la escritura más fácil y la tabla más fácil de leer, liste los valores iniciales en reset y entonces liste los valores de los registros y Latch sólo cuando una carga a registro ha sido realizada o cuando un Latch llega a ser transparente.

Escribir el Informe

Utilice la lista siguiente de temas para empezar su informe de proyecto en formato IEEE.

- Resumen
- Introducción
- Resumen de la trayectoria de datos – incluya la figura de la trayectoria de datos y tablas
- Resumen de la unidad de control – incluya la figura y tabla
- Breve explicación del micro código simbólico
- Lista simbólica para la demostración del código y discusión
- Contenido en el Microstore
- Tabla de acontecimiento – asegúrese resumir la convención usada para hacer la tabla de acontecimiento más legible.
- Conclusiones Generales
- Inserte en un apéndice los resultados actuales de simulación, con un título apropiado
- Apéndice para código fuente. Liste en una primera página todos los módulos verilog y en las siguientes páginas las impresiones del código verilog actual. Como siempre incluya un bloque de comentario a la cabeza de cada archivo que proporcione el nombre del archivo, de su nombre, de la fecha y de una declaración significativa breve acerca de lo que el archivo contiene
- Inserte en un apéndice un comentario junto con una lista de los cambios que usted hizo al archivo CST o UCF.